

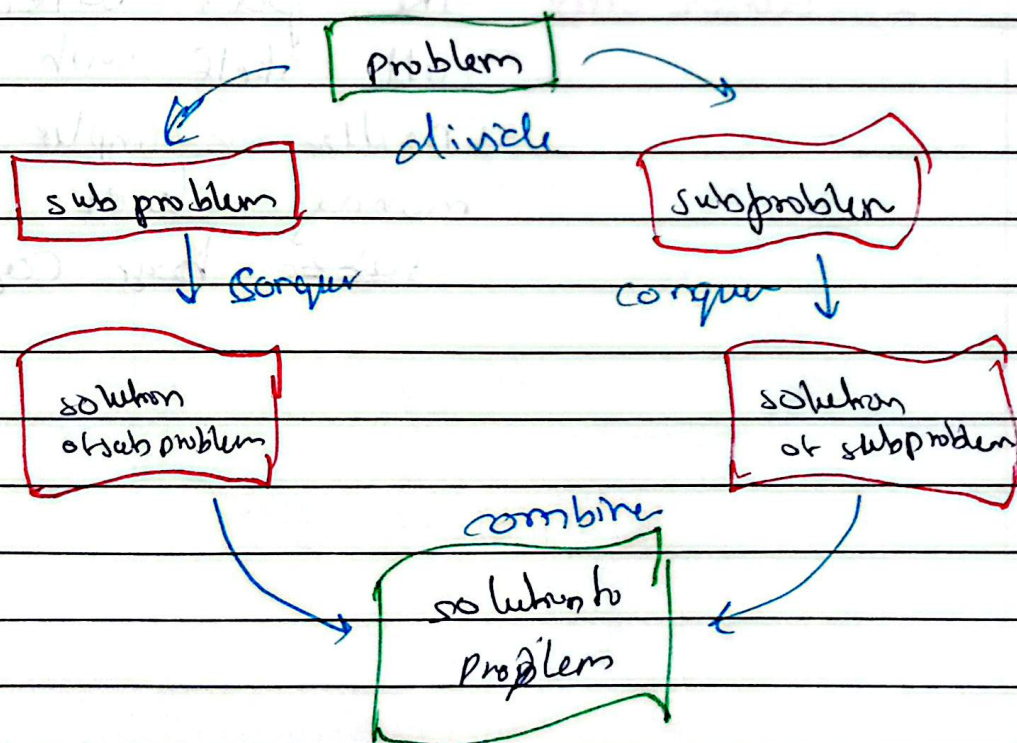
Recursion

Understanding Divide & Conquer:

Divide: the problem into smaller subproblems that are smaller instances of same problem

Conquer: the subproblems by solving them recursively.

Combine: the solutions to the subproblems into the solution for the original problem



Recursion:

It occurs when a **function / Procedure** calls it self.

There are 2 parts of any recursive function:

1) **Base case:** The simplest possible input where the answer is known directly without making further recursive calls.

2) **Recursive Step:** The part where the function calls itself with a strictly smaller or simpler subproblem moving closer to the ~~sub~~ Base Case.

/date

Factorial example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$4! = 4 \times 3 \times 2 \times 1 = 24.$$

Algo:

fact (n) {

if (n <= 1)

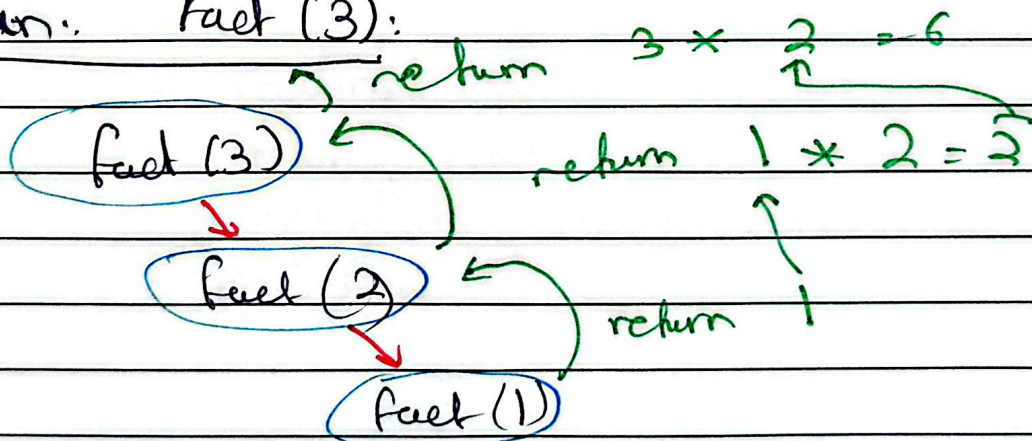
return 1;

} base case

return n * fact(n-1); } recursive step.

}

Dry run: fact (3):



~~fact(3)~~

BHATTI

Example 2: Fibonacci:

```
Fib(n) {
```

```
    if (n <= 2)
```

```
        return 1
```

```
    return fib(n-1) + fib(n-2)
```

```
}
```

} - Base
Case

} - recursive
step

Recursion problems:

* Stack Overflow:

- Recursive function cannot use statically allocated local variables
- Calling a recursive function many times or with large arguments may result in stack overflow.

Recursive definition in mathematical forms:

Now we will write the recursive functions in mathematical forms.

Also called "Recurrence relations"

- we translate the code directly into a piecewise function $T(n)$.

Ex. To make Recurrence relation from code, one needs to understand:

- Base Case
- Recursive Case.

Example 1: factorial:

fact (n) {

if (n <= 1)

return 1;

return n * fact(n-1) → recursive case

}

Base case shows:

if $(n \leq 0)$ then answer is 1.

$$1, n \leq 1$$

Recursive case:

$$n * f(n-1) \quad \text{if } n > 1$$

so

$$f(n) = \begin{cases} 1 & , n \leq 1 \\ n * f(n-1) & , n > 1 \end{cases}$$

Also from this we can derive Time complexity Recurrence:

$$T(n) = \begin{cases} O(1) & , n \leq 1 \\ \underbrace{T(n-1)}_{\text{recursive call}} + \underbrace{O(1)}_{\text{multiply operation}} & \text{if } n > 1 \end{cases}$$

Example 2:

Write Recursive relation & C++ code for counting digits eg:

$$19865 = 5, \quad 2316781 = 7$$

$$2478 = 4$$

Code/Algo

```
Count Digits (int num) {
```

```
    num/10 == 0
    if (num < 0) {
```

```
        return 1;
```

```
    }
```

```
    return countDigits (num/10) + 1;
```

```
}
```

Recurrence relation:

$$f(n) = \begin{cases} 1, & -10 < n < 10 \\ 1 + f\left(\left\lfloor \frac{n}{10} \right\rfloor\right), & n \geq 10 \end{cases}$$

Time &
Recurrence.

$$T(n) = \begin{cases} \Theta(1), & -10 < n < 10 \\ \Theta(1) + T\left(\frac{n}{10}\right), & n \geq 10 \end{cases}$$

Example 3:

Finding exponents $\cdot$ `int power (int k, int n)`
 find k^n .

BUT you are not allowed to do
~~do~~ $n-1$ in recursive call instead
 do $n/2$.

Code:

```
int power (n, k) {
    if (not n == 0) {
        return 1;
    }
    else {
        int t = power (n/2, k)
        if (n % 2 == 0)
            return t*t;
        else
            return k * t * t;
    }
}
```

Recurrence:

$$S(n, k) = \begin{cases} \text{answer} & , n \geq 0 \\ S(k, n/2) + C & , n < 0 \end{cases}$$

we can
write C or
 $\Theta(1)$.

$$T(n) = \begin{cases} \Theta(1) & , n \geq 0 \\ S(n/2) + \Theta(1) & , n < 0 \end{cases}$$

~~removed~~ as well as
it was just an extra
term, not ~~really~~
affected by recursion

Example 3: (multiple recursive statements).

e.g. consider a code:

```
fun ( ... ) {
```

```
    base O(1) // base case
```

```
    return fun ( n/2 ) //
```

```
    return fun ( n/4 ) + n3 //
```

```
    return fun ( n/2 ) //
```

recursive cases

all 3.

```
}
```

I wrote a simple code but all these 3 statements should be reachable somewhere in code not like never. (or else we won't consider).

So for this our

$$T(n) = \begin{cases} O(1) \\ 2T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n^3. \end{cases}$$

as 2 statements $\frac{n}{2}$ & one $\frac{n}{4}$

BHATTI

/date

IMP:

To make the recurrence relations we
~~simply~~ can

* add up all the recursive
cases.

* ignore constants

* base case remains same.

BHATTI